

Hashfuncs

Fast fingerprints, buckets & cache keys.



Install & load

```
INSTALL hashfuncs FROM community;  
LOAD hashfuncs;
```

01 At a glance

Hash SQL values with named non-cryptographic algorithms. Choose a stable representation and seed for repeatable partitioning, probabilistic-filter inputs and compact fingerprints.

Canonical input → Named hash → Bucket / key

02 Core SQL surface

`xxh3_64(value [, seed])`
Fast 64-bit result as UBIGINT.

`xxh3_128(value [, seed])`
128-bit result as UHUGEINT.

`xxh3_128_hex(text [, seed])`
Canonical 32-character lowercase hex digest for strings and BLOBs.

`xxh32 / xxh64`
Classic xxHash variants, with optional seed overloads.

`rapidhash(value [, seed])`
64-bit RapidHash for general-purpose fingerprints.

`murmurhash3_32 / murmurhash3_x64_128`
Named MurmurHash3 variants for explicit algorithm selection.

03 Keep in mind

Keep the representation fixed

Match input bytes, type and seed. Fixed v1.5 hex output uses canonical high64 then low64. Older 0dec806 reverses those halves. Numeric UHUGEINT output retains its legacy layout for stored-key compatibility; use the hex function for interchange.

04 Route to 16 buckets

```
SELECT i, xxh3_64(i, 42) % 16 AS bucket  
FROM range(8) AS t(i);
```

The same input type and seed assign each value to the same bucket.

05 Format a string fingerprint

```
SELECT xxh3_128_hex('hello') AS digest;  
-- 32 lowercase hexadecimal characters
```

Fixed v1.5 builds match canonical xxHash digests directly.

06 Choose the output width

```
SELECT typeof(xxh3_64('hello')) AS small,  
       typeof(xxh3_128('hello')) AS large;  
-- UBIGINT, UHUGEINT
```

Choose the width for your collision budget and storage format.

07 In practice

Collisions are possible

Hashes are compact fingerprints, not uniqueness constraints. Retain original keys for exact equality. Use crypto for authentication or adversarial inputs.

[Full docs & examples ↗](#) [Source ↗](#)